EXP.NO: 10

FIFO&LRU Page Replacement

DATE: 17/04/2026

AIM:

To implement and compare page replacement algorithms — First In First Out (FIFO) and Least Recently Used (LRU) — and compute the total number of page faults for a given reference string and a fixed number of frames.

ALGORITHM:

1. Start the experiment.
2. Input the page reference string and the number of frames.
3. FIFO: Maintain a queue of pages currently in memory. On a page fault, replace the page that arrived first (front of the queue). Increment the page fault counter.
4. LRU: Track the most recent access time for each page in memory. On a page fault, replace the page that was least recently used. Increment the page fault counter.
5. For each page reference, determine whether it is a page hit (page already in frame) or a page fault (page not in frame).
6. Display the frame state after each reference along with hit/fault status.
7. Print the total number of page faults for both algorithms and compare.
8. Stop the experiment.

CODE:

```
#include <stdio.h>

#define MAX 20

/* ■■ FIFO Page Replacement ■■ */
void fifo(int pages[], int n, int frames) {
    int frame[MAX], queue[MAX];
    int front = 0, faults = 0, filled = 0;
    for (int i = 0; i < frames; i++) frame[i] = -1;

    printf("\n-- FIFO --\n");
    printf("Ref Frames                    Status\n");
    for (int i = 0; i < n; i++) {
        int found = 0;
        for (int j = 0; j < frames; j++)
            if(frame[j]==pages[i]){found = 1; break; }

        if (!found) {
            frame[front % frames] = pages[i];
            queue[front % frames] = pages[i];
            front++;
            if (filled < frames) filled++;
            faults++;
            printf(" %d |", pages[i]);
```

```
            for(int j = 0; j < frames; j++)
                frame[j]==-1 ? printf(" - ") : printf(" %d ", frame[j]);
            printf("| Fault\n");
        } else {
            printf(" %d |", pages[i]);
            for(int j = 0; j < frames; j++)
                frame[j]==-1 ? printf(" - ") : printf(" %d ", frame[j]);
            printf("| Hit\n");
        }
    }
    printf("TotalPage Faults (FIFO): %d\n", faults);
}

/* ■■ LRU Page Replacement ■■ */
void lru(int pages[], int n, int frames) {
    int frame[MAX], recent[MAX];
    int faults = 0, counter = 0;
    for (int i = 0; i < frames; i++) { frame[i] = -1; recent[i] = 0; }

    printf("\n-- LRU --\n");
    printf("Ref Frames                    Status\n");
    for (int i = 0; i < n; i++) {
        int found = -1;
        for (int j = 0; j < frames; j++)
            if(frame[j]==pages[i]){found = j; break; }

        if (found == -1) {
            /*FindLRUslot:empty first, then least recently used */
            int lruIdx = 0;
            for(intj=0;j<frames; j++) {
                if(frame[j]==-1) { lruIdx = j; break; }
                if(recent[j]<recent[lruIdx]) lruIdx = j;
            }
            frame[lruIdx] =pages[i];
            recent[lruIdx] = ++counter;
            faults++;
            printf(" %d |",pages[i]);
            for(intj=0;j<frames; j++)
                frame[j]==-1?printf(" - ") : printf(" %d ", frame[j]);
            printf("| Fault\n");
        } else {
            recent[found] = ++counter;
            printf(" %d |",pages[i]);
            for(intj=0;j<frames; j++)
                frame[j]==-1?printf(" - ") : printf(" %d ", frame[j]);
            printf("| Hit\n");
        }
    }
    printf("TotalPageFaults(LRU): %d\n", faults);
}

int main() {
    int pages[] = {7,0,1,2,0,3,0,4,2,3,0,3};
    int n = 12, frames = 3;
    fifo(pages, n, frames);
    lru (pages, n, frames);
```

```
    return 0;
}
```

OUTPUT:

```
[hp@localhost ~]$ gcc page_replace.c -o pr
[hp@localhost ~]$ ./pr

-- FIFO --
Ref Frames                  Status
 7 | 7 - - | Fault
 0 | 7 0 - | Fault
 1 | 7 0 1 | Fault
 2 | 2 0 1 | Fault
 0 | 2 0 1 | Hit
 3 | 2 3 1 | Fault
 0 | 2 3 0 | Fault
 4 | 4 3 0 | Fault
 2 | 4 2 0 | Fault
 3 | 4 2 3 | Fault
 0 | 0 2 3 | Fault
 3 | 0 2 3 | Hit
Total Page Faults (FIFO): 10

-- LRU --
Ref Frames                  Status
7 | 7 - - | Fault
 0 | 7 0 - | Fault
 1 | 7 0 1 | Fault
 2 | 2 0 1 | Fault
 0 | 2 0 1 | Hit
 3 | 2 0 3 | Fault
 0 | 2 0 3 | Hit
 4 | 4 0 3 | Fault
 2 | 4 0 2 | Fault
 3 | 4 3 2 | Fault
 0 | 0 3 2 | Fault
 3 | 0 3 2 | Hit
Total Page Faults (LRU): 9
```

RESULT:

Thus, FIFO and LRU page replacement algorithms were successfully implemented. LRU produced fewer page faults (9) compared to FIFO (10) for the given reference string, demonstrating that LRU performs better by replacing the least recently used page.